*Need help? Self-Service || Yammer || **No new spaces from Nov-23rd***

iOS Dev Guide - Create new TextInputHelper

Created by Peggy Liu, last modified on 18 Feb 2021

Information on using the TextField, click here

Creating a New TextInputHelper

If you need a new TextInputHelper that has validation rules not covered in any of the Existing InputHelpers, you can contribute one to the Able Library by inheriting the TextInputHelperProtocol and one of either TextValidationHelper or NumberValidationHelper

The process:

1. Get the details needed to create inputHelper e.g. Validation rules, text, error messages - check it with the Able Team
2. pull the latest able catalogue + able library
3. Create a new InputHelperType Class using this template below (and unit test) - add to Formatter if new Format is needed
4. Implement it and test it
5. When you are happy with it, create a pull request to the Able Repo
6. Notify the Able Team for it to be added to the Design System and made ready for the next Able Update Release

Videos describing below in Catalogue App to create the InputHelperSIM:

Part 1 - create and use InputHelper: https://web.microsoftstream.com/video/9d05d2f3-c23e-4854-8388-87fc6c5d4db0
Part 2 - customise Textfield text: https://web.microsoftstream.com/video/c45c6fae-5331-4494-8719-a936f801fb05

Sample InputHelper Template

```
import Foundation

import UIKit

// Must Inherit the TextInputHelperProtocol and one of either TextValidationHelper (ascii) or NumberValidationHelper (numb

public class InputHelperNEW: NumberValidationHelper, TextInputHelperProtocol {

    public var contentType: UITextContentType?

    public var topText: String

    public var helpText: String

    public var fieldText: String = ""


    public var attributed: NSAttributedString

    public var validationHandler: ValidationHandler

    public var inlineValidationHandler: ValidationHandler?



    //Constructor 1 - customise the textfield labels and error message dictionary here

       public init(validationHandler: @escaping ValidationHandler, isRequired: Bool = true) {

           self.validationHandler = validationHandler

               topText = "Label of the textfield"

               helpText = "Optional helpful piece of information about input or field here"


            self.attributed = NSAttributedString(string: self.fieldText)

           let validationDictionary: ValidationDictionary = [

                   .submitError1: "Back end validation failure 1",

                   .submitError2: "Back end validation failure 2",

                   .empty: Constants.TextFieldDefaultErrorMessage.requiredFieldMsg,

                   .invalidFormat: "Invalid length. Description of the error here"]

           super.init(validationDictionary: validationDictionary, maxCharacters: 13)

           isRequiredField = isRequired
        //optional params
```

```
        kernIndexes = [2] //indexes of where chunking occurs
        shouldChunkOnEdit = true //if we want the chunking to happen while editing
        replaceAttributesOnEndEditing = false // default is false - only true for when tapping out we want to replace the

       }



    //Constructor 2

    public init(topText: String,

                helpText: String,

                validationDictionary: ValidationDictionary,

                validationHandler: @escaping ValidationHandler,

                isRequired: Bool = true) {

           self.topText = topText

           self.helpText = helpText

            self.fieldText = ""

           self.validationHandler = validationHandler


            self.attributed = NSAttributedString(string: self.fieldText)
        super.init(validationDictionary: validationDictionary, maxCharacters: 13)

        isRequiredField = isRequired


        //optional params
        kernIndexes = [2] //indexes of where chunking occurs
        shouldChunkOnEdit = true //if we want the chunking to happen while editing
        replaceAttributesOnEndEditing = false // default is false - only true for when tapping out we want to replace the

    }

    public override func validateInline(proposedInput: String) -> Bool {

        //If you want to override the Inline validation of characters supplied by

        //TextValidationHelper (ascii characters) and NumberValidationHelper (numbers only)

        // so override only if your Inputhelper has extra validation of Inline rules

        //Returns a bool to indicate Valid Input (True) or Invalid Input (False)

    }

    // This is the validation that happens on the textfield after losing focus

    public func validateEndValue(inputValue: String) -> ValidationResult {

        hasValidatedOnce = true

        if let passrequired = requiredFieldValidation(inputValue: inputValue) {

            return passrequired

        } //this is the check for optional or required field, do not edit



        //customise below to do needed validations and return the corresponding ValidationResult

        if inputValue.count != maxCharacters || !validateInline(proposedInput: inputValue) {

            return .failure(.invalidFormat)

        } else { //success
```

```
            return .success(inputValue)

        }

    }

    //This is for any formatting of the Input Value you need for the TextField e.g. 000 000

    public func format(value: String) -> String {

        return value  // if no change needed

        //Sample formatting of mobile number:
       let formatter = Formatter() <- using a formatter object in Able, create as needed for any new formatTypes
        return formatter.formatNumberString(numberString: value, formatType: .australianmobile)

    }

}
```

Sample Unit Test

```
@testable import Able

import XCTest

class InputHelperSIMTests: XCTestCase {

    var inputHelper: InputHelperSIM!

    var inputHelperOptional: InputHelperSIM!

    override func setUp() {

      inputHelper = InputHelperSIM { _ in

      }

      inputHelperOptional = InputHelperSIM(isRequired: false) { _ in

      }

    }

    func testValidateEndValue() {

       let result = inputHelper.validateEndValue(inputValue: "letters that should not be here")

        XCTAssertEqual(result, .failure(.invalidFormat))

        let result2 = inputHelper.validateEndValue(inputValue: "2323345674678")

        XCTAssertEqual(result2, .success("2323345674678"))

        let result3 = inputHelper.validateEndValue(inputValue: "249")

        XCTAssertEqual(result3, .failure(.invalidFormat))

        let result4 = inputHelper.validateEndValue(inputValue: "")

        XCTAssertEqual(result4, .failure(.empty))

        let result5 = inputHelperOptional.validateEndValue(inputValue: "")

        XCTAssertEqual(result5, .success(""))

        let result6 = inputHelperOptional.validateEndValue(inputValue: "04942356435345345433423")

        XCTAssertEqual(result6, .failure(.invalidFormat))

        let result7 = inputHelperOptional.validateEndValue(inputValue: "5")
```

```
        XCTAssertEqual(result7, .failure(.invalidFormat))

        let result8 = inputHelper.validateEndValue(inputValue: "04rt")

        XCTAssertEqual(result8, .failure(.invalidFormat))

    }

    func testValidateInline() {

        XCTAssertFalse(inputHelper.validateInline(proposedInput: "a"), "invalid a")

        XCTAssertFalse(inputHelper.validateInline(proposedInput: "ads234@#%@"), "invalid ads234@#%@")

        XCTAssertTrue(inputHelper.validateInline(proposedInput: "5"), "valid 5")

        XCTAssertTrue(inputHelper.validateInline(proposedInput: "5676"), "valid 5675")

    }

    func testCleancopied() {

        XCTAssertEqual(inputHelper.cleanCopied(proposedInput: "dfgvvf29853#%@$%@"), "29853")

        XCTAssertEqual(inputHelper.cleanCopied(proposedInput: "randomñó34"), "34")

    }

    func testGetValidationMessage() {

        XCTAssertEqual(inputHelper.validationMessage(for: .failure(.empty)), "Required field.")

        XCTAssertEqual(inputHelper.validationMessage(for: .failure(.submitError1)), "Back end validation failure 1")

        XCTAssertEqual(inputHelper.validationMessage(for: .failure(.submitError2)), "Back end validation failure 2")

        XCTAssertEqual(inputHelper.validationMessage(for: .failure(.invalidFormat)),

                        "Invalid length. A SIM number is 13 digits found on the back of the SIM.")

    }

    func testCustomText() {

        inputHelper = InputHelperSIM(topText: "custom topText", helpText: "custom helpText", validationDictionary: [

        .submitError1: "Back end validation failure 1",

        .submitError2: "Back end validation failure 2",

        .empty: Constants.TextFieldDefaultErrorMessage.requiredFieldMsg,

        .invalidFormat: "Invalid Format."]) { _ in

        }

        XCTAssertEqual(inputHelper.topText, "custom topText")

        XCTAssertEqual(inputHelper.helpText, "custom helpText")

    }

}
```

New Formatter

```
//create new format in Formatter e.g.
private func formatBankAccNum(numberString: String, charCount: Int) -> String {
        var formatted = numberString
        switch charCount {
        case 4, 5, 6, 7:
            formatted.insert(" ", at: numberString.index(numberString.startIndex, offsetBy: 3))
            return formatted
        case 8:
```

```
            formatted.insert(" ", at: numberString.index(numberString.startIndex, offsetBy: 4))
            return formatted
        case 9, 10:
            formatted.insert(" ", at: numberString.index(numberString.startIndex, offsetBy: 3))
            formatted.insert(" ", at: numberString.index(numberString.startIndex, offsetBy: 7))
            return formatted
        default:
            return numberString
        }
    }


//dont forget to add this type to the cases in public enum NumberFormatType and in the func formatNumberString(numberStrin


//Add new formatter method in FormatterTests
//e.g.
func testFormatABN() {
       XCTAssertEqual("33 051 775 556", formatter.formatNumberString(numberString: "33051775556", formatType: .abn))
       XCTAssertEqual("33 051 775 55", formatter.formatNumberString(numberString: "3305177555", formatType: .abn))
       XCTAssertEqual("33 051 77", formatter.formatNumberString(numberString: "3305177", formatType: .abn))
       XCTAssertEqual("33 05", formatter.formatNumberString(numberString: "3305", formatType: .abn))
       XCTAssertEqual("33 0", formatter.formatNumberString(numberString: "330", formatType: .abn))
    }
```

No labels

Atlassian